Fuzzing for Web Browser Under-Invalidation Bugs

Anonymous Submission

***Abstract*—Fuzzing for web browsers has traditionally focused on crash testing, such as for finding security-relevant bugs in parsers and media processing engines, and regression testing, such as for finding newly-introduced bugs in rendering code. However, fuzzing has not traditionally been used for finding correctness bugs in the implementation of the web platform standards, because writing effective test oracles for these standards is challenging. We introduce a class of correctness bugs that afflict web browser layout engines: under-invalidation bugs, which cause browsers to incompletely update internal state in response to page updates.**

To combat these bugs, we introduce LAYOUTQUICKCHECK, a fuzzer for finding, minimizing, and clustering under-invalidation bugs. LAYOUTQUICKCHECK uses the browser as its own oracle, comparing an incremental and from-scratch layout of the same web page and flagging differences as under-invalidation bugs. LAYOUTQUICKCHECK reproduces XXX bugs from Chrome and Firefox's public bug trackers and finds XXX novel bugs that are reported to and accepted by developers. Moreover, LAYOUTQUICKCHECK's minimization and clustering tooling reduces large generated failures into smaller, developer-actionable reports.

I. INTRODUCTION

Web browsers are central to modern computing, with browser-based usage corresponding to over **80%** of daily work in some organizations [1]. Browser makers therefore invest heavily in fuzzing, audits, security research, standardization, shared test suites, and interoperability efforts aimed at ensuring browser correctness [**?**], [**?**], [**?**], [**?**], [**?**], [**?**].

Developers increasingly use *fuzzing*, an automated testing technique that exercises software with large numbers of randomly generated or mutated inputs, to identify and fix bugs. Browser fuzzers have been successful at finding crashes, security vulnerabilities, and rendering regressions. R2Z2 detects rendering regressions by fuzzing the output of different browser versions [2]; Metamong detects render-update bugs through fuzzing [3]; and FuzzOrigin targets UXSS vulnerabilities through origin fuzzing [4]. However, one important browser mechanism remains difficult to test systematically: *layout invalidation*, which determines which parts of a page must be recomputed after changes to the DOM or CSS.

Layout invalidation is difficult because browsers attempt to reuse as much previous work as possible while still ensuring that the final page state is correct. The browser rendering pipeline includes parsing, style computation, layout, painting, and related stages [**?**]. After a DOM or CSS update, invalidation decides which work can be reused and which work must be recomputed. Public browser-engine trackers contain invalidation failures in Chromium, Firefox, and WebKit-based browsers [5]–[10]. Modern layout engines, including Chrome's LayoutNG, are designed in part to reduce these correctness bugs [11], and some optimizations have historically stalled because ensuring correct invalidation was too difficult [12].

This paper introduces LAYOUTQUICKCHECK, a fuzzer for detecting under-invalidation bugs in modern web browser engines. LAYOUTQUICKCHECK systematically exercises incremental layout behavior by generating DOM and CSS updates that should force the browser to recompute affected rendering state. Its central strategy is self-comparison: the system compares the browser's incremental layout result against its own from-scratch layout result for the same final page. This allows LAYOUTQUICKCHECK to detect stale layout state without relying on a separate browser implementation.

This paper makes the following contributions:

- We describe the challenges of extending fuzzing to browser layout invalidation and explain why existing browser fuzzers do not systematically target under-invalidation bugs.
- We design LAYOUTQUICKCHECK, a targeted fuzzer that generates DOM and CSS updates, compares incremental and fresh layout results, and reports stale layout or rendering state.
- We evaluate LAYOUTQUICKCHECK through experiments and ablation studies across XXX browser engines, measuring bug discovery, minimization, and clustering effectiveness.
- We release LAYOUTQUICKCHECK together with evaluation artifacts, generated test cases, and benchmarks at the following URL: XXX.

II. BACKGROUND, RELATED WORK, AND MOTIVATION

This section introduces the fundamental topics related to LAYOUTQUICKCHECK: web browsers, under-invalidation bugs, and the challenges of fuzzing browser layout engines.

A. Web Browsers

Web browsers are among the most widely used and economically important software systems. Their engines parse web content, compute style and layout, execute scripts, and paint pixels to the screen. Common examples include Chromium's Blink engine [13], Mozilla's Gecko engine [14], and Apple's WebKit engine [15]. Browser engines require large sustained investment; for example, Open Web Advocacy estimates that Google provides roughly 90% of Chromium development funding and invests around $1 billion annually in the web platform [16].

Unlike many traditional fuzzing targets, web browsers process highly structured inputs made up of HTML elements, CSS style rules, and JavaScript updates. These inputs can exercise

DOM parsing, selector matching, style resolution, layout, and invalidation [17], [18]. Effective browser fuzzing therefore requires more than generating valid syntax: the generated inputs must be optimized toward the specific class of bugs being tested [19]–[21].

After a page has loaded, browsers rarely recompute the entire page for every change. Instead, they use invalidation logic to decide which parts of the rendering state are affected by a DOM or CSS update.[TODO: Ref?] Correct invalidation preserves the performance benefits of incremental rendering while ensuring that the final pixels match the page's current state [17], [22]. Over-invalidation recomputes too much and hurts performance, while under-invalidation recomputes too little and leaves stale layout or rendering state visible [11], [18], [22].

B. Why Fuzzers Fail to Find Under-Invalidation Bugs

Most browser fuzzers are optimized for bug classes with explicit failure signals. JavaScript-engine fuzzers such as CodeAlchemist and DIE generate complex programs to expose crashes and security vulnerabilities [23], [24]. Other browser fuzzers target specialized attack surfaces such as WebGL or WebAssembly, where crashes, memory corruption, or API errors provide concrete oracles [25], [26]. These approaches are valuable, but they do not directly test whether a layout engine correctly updates stale rendering state after a DOM or CSS change.

Rendering-engine fuzzers begin to address this issue, but their goals still differ from detecting under-invalidation. RT-Fuzz mutates render-tree state but primarily evaluates robustness defects such as crashes [27]. Differential rendering fuzzers such as R2Z2, Metamong, and JANUS compare outputs across versions, executions, or browsers [2], [3], [28]. These oracles are useful, but they do not fully solve layout under-invalidation because cross-version or cross-browser differences still require determining which result is correct. Layout under-invalidation therefore needs a more targeted oracle.

C. Why it is Difficult to Isolate Under-Invalidation Bugs?

Finding an under-invalidation symptom is only the first step. Isolating the underlying bug is difficult because incremental layout is a cache-like optimization: the browser attempts to reuse previous layout results and recompute only the parts affected by a change [29]. A stale layout result can depend on a specific DOM tree, style set, mutation, and internal invalidation decision. If an irrelevant-looking part of the generated test case is removed too early, the browser may take a different invalidation path and the bug may disappear.

The complexity of layout engines also makes bug reports hard to reduce to their root cause. Prior work observes that correct incremental layout requires reasoning about when recomputation can safely be skipped [30]. As a result, many generated tests can expose the same underlying invalidation mistake through different combinations of elements, CSS properties, and mutations.

These properties make both minimization and deduplication necessary. A useful bug report should remove unrelated elements, styles, and mutations while preserving the incremental-vs-fresh mismatch [31]. For HTML/CSS tests, reduction also needs to respect document structure so minimized tests remain valid and meaningful [32], [33].

Motivation: the need for an under-invalidation browser layout fuzzer. Existing browser fuzzers struggle to detect layout under-invalidation bugs. LAYOUTQUICKCHECK addresses this gap by making browser layout invalidation testable with a targeted oracle, minimization, and deduplication.

III. CHALLENGES AND SOLUTIONS

To address the limitations of conventional browser fuzzing for layout invalidation correctness, we introduce LAYOUTQUICKCHECK: a system for finding, minimizing, and grouping under-invalidation bugs in modern web browser layout engines. In the following, we outline core challenges that shape LAYOUTQUICKCHECK's design, along with their solutions aimed at effectively fuzzing browser layout invalidation.

A. Challenge 1: Generating Inputs that Trigger Under-Invalidation Bugs

The first challenge is generating inputs that are both valid web pages and useful tests for layout invalidation. A browser fuzzer can easily[TODO: Ref] create HTML and CSS that the browser accepts, but such inputs may never exercise the incremental layout paths where under-invalidation bugs occur. To expose these bugs, the generated page must include an initial layout, a meaningful DOM or CSS update, and a final state where the browser should recompute affected geometry. LAYOUTQUICKCHECK therefore biases generation toward CSS properties and mutations that are likely to change layout, so that each test is more likely to trigger the browser's invalidation logic rather than only testing parsing or general rendering behavior.

Display: The `display` property is a useful target because it determines the layout mode used for an element and can also change how its children participate in layout.[TODO: Ref?] Changing `display` can move an element between block, inline, flex, grid, table, or hidden states, each of which requires different layout behavior. Because these changes can affect both the modified element and its descendants, they provide strong opportunities to expose missed invalidation decisions.

Grid: Grid layout creates many invalidation opportunities because it introduces relationships between a container and its children.[TODO: Ref?] Changing grid templates, gaps, placement rules, or auto-flow can reposition multiple children from a single style update. This makes grid a strong target for LAYOUTQUICKCHECK because a missed invalidation can leave incorrects child positions or incorrect container geometry.

Flow-root: The `display:flow-root` value is important because it changes layout boundaries and reflow-root behavior. **[TODO: Add more about flow-root.]**

Width and height: Width and height changes directly affect an element's box geometry. When an element changes size, the browser may need to recompute that element, its descendants, siblings, and sometimes ancestors. These mutations are useful because layout engines often appears as an obvious mismatch in element size or position.

Percentage and pixel sizes: Percentage sizes test parent-dependent layout behavior because the final size of an element depends on its containing block. Pixel sizes create more direct change to the geometry possibly causing conflict issues with surrounding elements. Using both lets LAYOUTQUICKCHECK exercise dependent layout constraints to produce failures that are easy to observe and reproduce.

Transforms: Transform changes affect visual position and reported geometry. **[TODO: Add more about transforms.]**

Noise reduction: LAYOUTQUICKCHECK disables properties such as `writing-mode`, `content-visibility`, padding, and table display values because they are less likely to produce under-invalidation bugs. Instead, these properties often add unnecessary complexity to the generated tests.**[TODO: Ref?]** Removing them keeps the generated tests focused on mutations that are more likely to expose missed invalidation decisions and helps produce smaller, clearer bug reports.

Solution 1: LAYOUTQUICKCHECK includes a generation strategy for steering generated inputs toward page updates that are likely to trigger layout under-invalidation bugs. The system focuses on CSS properties and mutations that should force layout recomputation, increasing the chance that a missed invalidation decision becomes observable.

B. Challenge 2: Minimizing and Isolating Under-Invalidation Bugs

Under-invalidation bugs often appear in large generated pages that contain many elements, styles, and mutations.**[TODO: Ref?]** Even when only a small part of the page is responsible for the failure, the original test case may include many unrelated structures introduced by random generation. These large inputs are difficult for developers to inspect and make it hard to identify which DOM structure or CSS change actually triggered the stale layout state.

Preserving the bug trigger: Minimization is difficult because under-invalidation bugs depend on a precise sequence of layout states. The browser first lays out the original page, then applies a DOM or CSS update, and then decides which cached layout information can be reused. If the minimizer removes an element, style, or mutation that seems unrelated but changes this invalidation path, the bug may disappear even though the original conditions were present.

Solution 2: LAYOUTQUICKCHECK reduces bug-triggering inputs to their essential elements, isolates the structures that cause the failure, and produces a test case that is easier to analyze and debug.

C. Challenge 3: Grouping and Categorizing Under-Invalidation Bugs

A major challenge in layout fuzzing is that many generated inputs can trigger the same underlying browser bug.**[TODO: Ref]** Two test cases may differ slightly in the HTML or CSS while still exercising the same missed invalidation decision inside the layout engine. Without grouping, developers would need to inspect many redundant reports before finding the distinct bugs that actually require attention.

Structural representation: Grouping under-invalidation bugs requires representing the parts of a test case that matter for the failure. The system must capture the DOM structure, base styles, modified styles, and the element relationships involved for each layout state. A consistent representation makes it possible to compare bugs by their underlying layout pattern rather than by superficial differences in generated HTML.

Automated categorization: Categorization is difficult because the process must be automated while still avoiding overly broad groups.**[TODO: Ref?]** If the grouping rule is too strict, the system leaves many duplicates unmerged; if it is too loose, unrelated bugs may be combined into the same category. LAYOUTQUICKCHECK therefore needs grouping logic that can recognize repeated bug patterns while preserving enough detail to keep distinct failures separate.

Developer-focused triage: The goal of grouping is to reduce the amount of manual triage required after fuzzing. Browser developers need to see recurring, actionable patterns rather than a long list of similar generated files. Categorized reports help developers focus on the likely root causes and decide which failures represent unique bugs.

Solution 3: LAYOUTQUICKCHECK groups bugs using structural rules and merged bug representations. This reduces duplicate reports, keeps similar failures together, and lets developers focus on the distinct under-invalidation patterns that are most likely to be actionable.

[TODO: Where to add that by grouping we can also bypass minimization or is that in implementation?]

IV. IMPLEMENTATION

LAYOUTQUICKCHECK is implemented in Python as an end-to-end, feedback-free fuzzing pipeline for browser layout invalidation. The implementation separates a test case from its serialization, executes the serialized page in a real browser through Selenium WebDriver. A under-invalidation failure is identified when there is a discrepancy between incremental and fresh layout computations. The same pipeline then classifies, minimizes,and sorts the result. At a high level, a run has six stages: (1) configuration and browser selection, (2) generation of a DOM and two style states, (3) execution of the layout oracle, (4) minimization of a bug-triggering test case, (5) optional structural grouping and minimization skipping, and (6) report and metric production. The following sections describe these stages and the concrete artifacts exchanged between them.

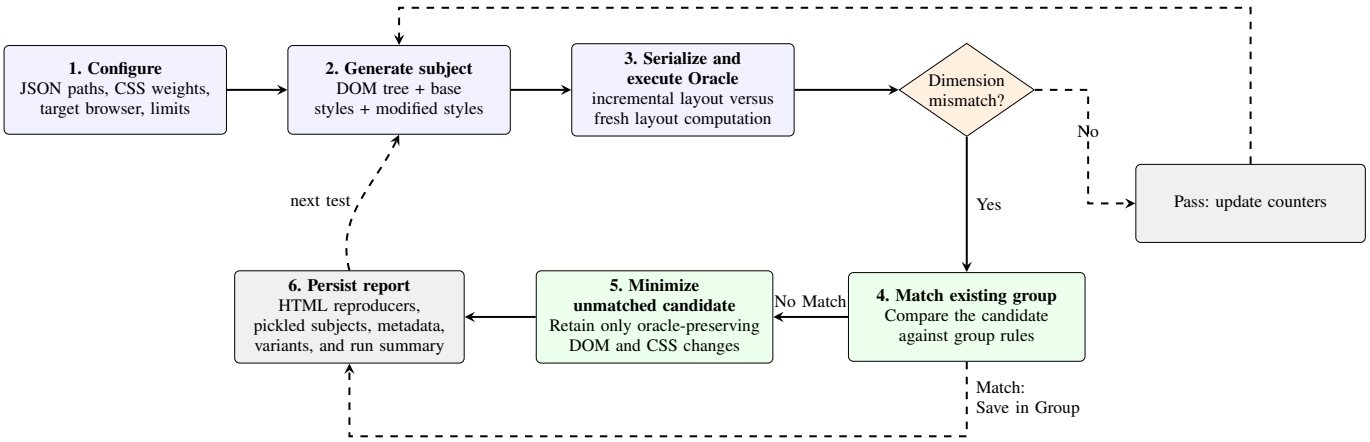


Fig. 1: LAYOUTQUICKCHECK pipeline. Each generated subject is tested by a browser-resident layout oracle. Passing subjects advance the fuzzing loop; candidates are structurally classified, minimized when necessary, and persisted with their reproduction artifacts and cumulative metrics.

A. Configuration, Execution Model, and Test Subjects

The command-line runner accepts a JSON configuration file and initializes a process-wide configuration object before any pages are generated. The configuration supplies two paths: a directory for persistent bug reports and a directory for temporary generated pages. It also supplies a list of browser variants and a map from CSS property names to generation weights. The runner supports a bug limit, test limit, and crash limit. The bug limit stops the experiment when it reaches the specified cumulative number of bug reports, while the test limit stops it at the specified cumulative number of executed test cases. The crash limit stops the current process after the specified number of browser or WebDriver failures; crashes from an earlier resumed process remain in the reported total but do not consume the new process's allowance. The `--no-sort` option disables structural classification and grouping, causing each candidate to be fully minimized and saved as an individual report rather than compared with existing bug groups. With no explicit limit, the runner remains active until it is interrupted or the crash limit is reached.

A browser variant specifies a browser type together with its WebDriver path, viewport, and optional browser settings. After reading a variant from the configuration, LAYOUTQUICKCHECK builds the corresponding Selenium WebDriver for Chrome, Firefox, or Safari. The variant marked `target` is used for the main fuzzing loop. Internally, each generated test is represented as a `RunSubject`:

- an `ElementTree`, which is a nested representation of the generated DOM;
- a base `StyleMap`, containing declarations applied in the initial document; and
- a modified `StyleMap`, containing declarations installed later by JavaScript.

Both style maps are indexed by generated element identifier and then by CSS property name. Keeping base and modified declarations separate captures the state transition being tested: the initial layout is computed from the base map, and the incremental update is the modified map.

B. DOM and CSS Generation

LAYOUTQUICKCHECK generates a body subtree recursively. The current generator uses `div` elements and text nodes. Each `div` receives a unique alphabetic identifier derived from a UUID so that it can be addressed from both CSS and the generated JavaScript. Generation is probabilistic: the body is very likely to receive children, while nested `div`s have a smaller probability of receiving one or more children. Text nodes contain one or more generated sentences, optionally separated by a newline. This produces trees with both nested boxes and text-dependent sizing behavior.

For every non-text element, the generator independently constructs base and modified style maps. It walks a repository-maintained catalogue of CSS properties and their supported value generators. The configured weight is first constrained to the range 0–100 and then divided by 100 to obtain the probability of selecting that property, i.e., $P(\text{select}) = w/100$. Thus, a weight of zero disables the property, a weight of 40 selects it with probability 0.40, and a weight of 100 selects it whenever it is considered. This random decision is made independently for each non-text element and for each of its base and modified style maps. If selected, a property-specific generator produces a valid length or keyword value. Unspecified properties use a default selection weight, allowing an experiment to focus on selected properties without having to enumerate the entire catalogue.

This design makes property weighting an input-distribution control rather than a change to the oracle. A configuration can emphasize properties that are relevant to a particular layout behavior while reducing or disabling the selection of less relevant properties. The weighted and unweighted experiments in Section V therefore execute the same generation and oracle pipeline but sample different style distributions.

C. Page Serialization and the Layout Oracle

Each `RunSubject` is serialized into a self-contained temporary HTML file. The serializer recursively emits the element tree, places base declarations in each element's inline `style` attribute, and emits a JavaScript `makeStyleChanges()` function. That function assigns every declaration in the modified style map through the element's `style` object. The generated page also includes a small JavaScript support file that implements the geometry collection and comparison operations. The runner uses a temporary HTML file to execute each generated test case.

The oracle compares two executions that should have final markup and style declarations an incremental and a fresh layout computation:

1) Selenium navigates the target browser to the generated local file URL and waits for the document body to be present.
2) `makeStyleChanges()` applies the modified declarations to the live document. This is the incremental path whose invalidation behavior is under test.
3) The page records `getBoundingClientRect()` for every DOM element. Each observation includes the element identifier, tag name, and its dimensions.
4) The page assigns the document element's `innerHTML` property to itself. This reconstruct forces the the browser to establish a fresh layout for that final state again.
5) The page collects the dimensions again and compares the two sets. A difference in `x`, `y`, `left`, `right`, `top`, `bottom`, `width`, or `height` for the same element is returned to Python.

The browser-side comparison returns a list of differing elements. Each entry identifies the affected element and records the attributes that differ as well as the values after incremental application and after document reconstruction. An empty list is a pass; a nonempty list is represented as a layout-bug result. WebDriver failures produce a separate page-crash result, while a page-load timeout is reported by the harness. Recording the individual geometric fields is useful during triage because it distinguishes, for example, a stale width from a descendant-position mismatch.

The oracle detects a candidate failure when the incremental and reconstructed layout paths produce different element dimensions in the selected browser and viewport.

D. Fuzzing Loop and Safe Repository

The runner first constructs a small repository of known-safe subjects. It generates and executes subjects until the configured safe directory contains 25 passing serialized `RunSubject` objects. The configured report directory is a persistent, cumulative bug repository: it contains all saved reports, groups, safe samples, and the run summary for that configuration. When a later invocation uses the same directory, it continues the same repository and aggregates newly discovered reports with the existing ones rather than creating a separate collection. These safe samples are used to prevent the grouping algorithm from accepting a rule that would also match known non-bug inputs.

The main loop then generates one subject, executes the oracle, increments the pass counter for a clean result, and enters post-processing only for a candidate. After every completed test, the runner updates the run summary. The summary records executed tests, passes, candidate bugs, failures that no longer reproduce during reduction, reports with no remaining modified style, crashes, elapsed time, and cumulative minimization and sorting time. It also counts standalone reports, grouped reports, and groups by scanning the report directory. **[TODO: Not in final version]** Threshold snapshots are written at every ten candidates and every 60 seconds of accumulated runtime, which supports time-series plots.

The runner resumes these cumulative statistics if a report directory already contains `run_summary.json`. Resuming with the same directory retains the existing bug reports, bug groups, grouping rules, and safe samples. New reports are therefore compared with the accumulated repository and can join an existing group instead of forcing the grouping process to start again. This makes interrupted long-running experiments continue as one statistical run, without losing their prior discovery, grouping, or measurement state.

E. Delta Debugging

Candidate pages are usually much larger than the condition required to reproduce a mismatch. LAYOUTQUICKCHECK applies delta debugging to reduce a bug-triggering test case while preserving the oracle result. For each proposed simplification, it deep-copies the current subject, executes the candidate in the target browser, and retains the copy only if the result remains a bug. Every accepted reduction becomes the new subject, so later reductions are tested against the smallest reproducer found so far. The delta-debugging process uses the following proposal families in order:

1) remove each element, including its associated base and modified style entries;
2) remove every style map for one element at a time;
3) remove individual base or modified declarations;
4) move one modified declaration into the base map, thereby testing whether that declaration must be an incremental change rather than merely being present in the final state;
5) replace generated length values with `20px` or `-20px`, preserving only their sign; and
6) apply readability-oriented transformations, including minimum box sizes, background colors, short identifiers, and an optional in-page control overlay.

The fourth step, implemented by `Minify_MoveStyleChangesToFirstLoad`, is particularly important for this bug class. If moving a declaration to the initial state preserves the mismatch, it is not necessary for the incremental transition and can be removed from the set of modified properties. Conversely, declarations that cannot be moved identify the state change that is more likely to matter to invalidation. The final oracle run is recorded again after all proposal families have been exhausted. If the candidate no longer reproduces, it is counted

as a false positive rather than saved as a confirmed layout report. A layout mismatch with an empty modified-style map is also counted separately because it no longer expresses the intended incremental CSS-change scenario.

F. Structural Grouping and Minimization Skipping

Full reduction is expensive when many generated subjects trigger a pattern already represented in the report directory. With sorting enabled, LAYOUTQUICKCHECK therefore attempts to match a new candidate against existing bug groups before full minimization. A subject is converted to a tree in which each node contains its tag, identifier, base styles, modified styles, and children; text is represented explicitly. The first node with modified styles is used as the start node for a rule because it anchors comparison on the update of interest. The rule represents the DOM structure relative to this anchor: it compares the modified node and its ancestor/descendant structure, rather than depending on that node's absolute location in the full page tree. This allows structurally similar update patterns to match even when they occur in different generated pages.

A group stores a merged representative tree and an extracted JSON rule. To form a candidate rule, the implementation merges two trees recursively. Equal tags, values, and declarations are retained. Disagreements are represented by the wildcard value `diff`. The resulting rule consists of an ordered HTML pattern and the common base and modified style requirements at the start node. Matching checks both components: the incoming subject must contain the ordered structural pattern and at least one matching element must have the required base and modified declarations. Wildcards permit variation where the merged reports disagree while retaining the shared structure and update signal.

Before accepting either a match or a new group, the rule is validated against the safe repository. Rules with no modified-style requirement or with a trivial single-element pattern are rejected, and a rule is rejected if it matches any safe sample. This is a conservative guard against a broad structural description that would classify ordinary pages as bug instances. It does not prove semantic equivalence, but it makes group promotion depend on both bug examples and known non-bug examples.

If an incoming candidate matches an existing rule, the runner skips the full reduction and saves it as another instance under that group. Otherwise, it minimizes the candidate first. The final minimized subject is then compared against existing groups and against standalone reports. Matching a group adds the new instance to that group. Matching a standalone report promotes the two reports into a newly created `bug-group-*` directory and writes the extracted rule. After every group update, group artifacts are recomputed so the merged representative and rule correspond to the instances currently stored in that directory. The sorting time is measured separately and added to the run summary alongside minimization time; this makes the cost and benefit of grouping measurable in Section V.

G. Report Layout, Reproduction, and Variant Results

Each saved report is placed in either a standalone `bug-*` directory or a child directory of a `bug-group-*` directory. To keep the bug repository lightweight, the report stores pickled forms of the pre-reduction and minimized `RunSubject`, rather than retaining separate HTML versions of both pages. These structured artifacts preserve the DOM and style states needed to rerun reduction or recompute grouping rules.

The accompanying `data.json` records the save timestamp, bug type, all styles used, base and modified style names, the geometric differences returned by the oracle, whether minimization was skipped, the matched rule, and the time spent sorting and minimizing. After minimization, the runner executes each configured browser variant and records browser name, browser version, operating-system details, viewport, variant description, and whether that variant also detects the bug. These fields distinguish the target run from supplementary checks and make reports usable when browser versions or command-line flags change.

Together, these artifacts allow a report to be inspected at several levels: the original input, the minimized browser reproducer, the exact geometry disagreement, the structured subject used by automated tooling, and its relationship to a group. This is the implementation basis for the throughput, bug-rate, minimization-time, and grouping measurements reported in the evaluation.

V. EVALUATION

Our evaluation of LAYOUTQUICKCHECK's browser fuzzing capabilities is guided by the following fundamental questions:

RQ1: How effective is LAYOUTQUICKCHECK at triggering an under-invalidation bugs?

RQ2: To what extent does sorting and clustering improve browser fuzzing?

RQ3: Is LAYOUTQUICKCHECK effective at finding new under-invalidation bugs?

Benchmarks: Table I summarizes the browser targets supported by LAYOUTQUICKCHECK. Our quantitative evaluation uses Chromium, which exercises the Blink layout engine, and Firefox, which exercises the Gecko layout engine. LAYOUTQUICKCHECK uses ChromeDriver and GeckoDriver, respectively.**[TODO: SAFARI DRIVER?]**

Browser	Rendering Engine	Driver
Chrome	Blink	ChromeDriver
Firefox	Gecko	GeckoDriver
Safari	WebKit	SafariDriver

TABLE I: Our browser-based layout fuzzing evaluation benchmarks.

Experiment Setup & Infrastructure: For RQ1, we evaluate four configurations in a fixed 60-minute execution window: Chromium with property weights, Chromium without property weights, Firefox with property weights, and Firefox without property weights. We record elapsed execution time from the beginning of each run and compare the number of bugs found, test-case throughput, and bug-detection rate within this

Browser	No weights			Weights		
	Speed	Bugs	Rate	Speed	Bugs	Rate
Chromium	225.3	115	0.847%	410.0	73	0.297%
Firefox	499.7	66	0.220%	497.6	140	0.469%

TABLE II: RQ1 bug-discovery results across browser and property-weight configurations over a 60-minute execution window. Speed is measured in executed tests per minute; rate is bugs divided by executed tests.

common budget. This design evaluates the effect of property weighting within each browser. For RQ2, we compare paired Firefox runs with sorting enabled and disabled over a nine-hour test run. At each elapsed-time point, we measure the cumulative time spent minimizing and sorting reports, as well as the cumulative number of executed test cases. Comparing both configurations over the same time window isolates whether sorting and clustering add post-processing overhead or reduce the work required for subsequent minimization, and whether that cost affects fuzzing throughput.

Post-processing Results: We evaluate each LAYOUTQUICKCHECK configuration using the following metrics: **Speed** via test-case throughput in executed tests per minute; **Bugs** via the cumulative number of reports that trigger the bug oracle; and **Rate** via the fraction of executed tests that trigger the oracle. We characterize post-processing using **Execution Time** via elapsed runtime, **Minimization and Clustering Time** via the cumulative time spent minimizing and clustering reports, and **Test Cases Executed** via the cumulative number of tests completed during the same interval. Finally, we report the numbers of bug groups and single reports after clustering. These counts are snapshots of report diversity and are not independently confirmed counts of root-cause bugs.

A. RQ1: Discovery of Under-Invalidation Bugs

To assess LAYOUTQUICKCHECK's ability to trigger under-invalidation bugs, we evaluate Chromium and Firefox with property weighting enabled and disabled. Property weighting biases generation toward selected CSS properties; the unweighted configuration samples from the same property space without this bias. We run each configuration for a fixed 60-minute window and compare its test-case throughput, cumulative bug reports, and bug-triggering rate. This comparison distinguishes configurations that find more reports because they execute more tests from those that find reports more frequently per test.

Figure 2 plots cumulative bug discovery and the corresponding bug-triggering rate over time, while Table II summarizes the final 60-minute results.

Outcomes: Property weighting has different effects across the two browser engines. In Firefox, weighting increases the number of bug reports from 66 to 140 and the trigger rate from 0.220% to 0.469%, while throughput remains nearly unchanged (499.7 versus 497.6 tests per minute). In Chromium, the unweighted configuration finds more reports (115 versus 73) and achieves a higher trigger rate (0.847% versus 0.297%). Within the firefox engine we have a significant jump in bug triggering rate with weighting.

RQ1: LAYOUTQUICKCHECK is effective at triggering under-invalidation bugs in both Chromium and Firefox. The weighted configuration finds more bugs in Firefox, while the unweighted configuration finds more bugs in Chromium. The effectiveness of property weighting is context-dependent and may vary with the specific browser and its layout engine.

B. RQ2: Cost of Sorting and Clustering

Minimizing every bug report produces a smaller, reproducible test case, but it is expensive when the fuzzer repeatedly triggers reports that belong to an existing bug group. Sorting and clustering are performed before full minimization to identify these repeated reports. When a report matches an existing group, LAYOUTQUICKCHECK can skip the full minimization phase and retain the existing minimized representative. We therefore compare paired Firefox configurations with sorting enabled and disabled over a nine-hour test run. We measure cumulative minimization and clustering time and the cumulative number of test cases executed to determine whether this skip mechanism saves post-processing time without reducing fuzzing throughput.

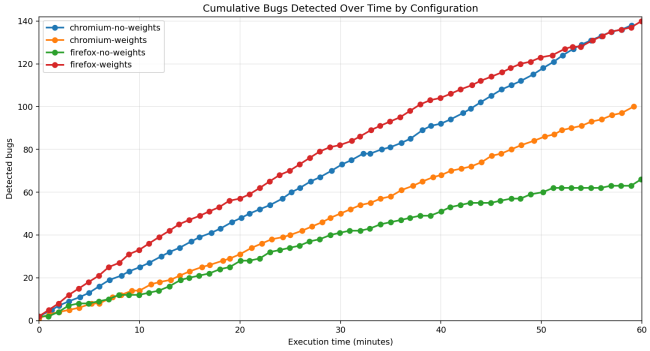
Outcomes: Figure 3 shows that sorting does not reduce test-case throughput in this experiment. At the common final snapshot of 32,040 seconds (8 hours and 54 minutes), the sorting configuration executes 130,714 test cases, compared with 126,325 without sorting. At the same snapshot, the sorting configuration accumulates 4,582.917 seconds of minimization and clustering time, whereas the no-sorting configuration accumulates 5,901.785 seconds. Thus, clustering reports before minimization is associated with 22.3% less cumulative post-processing time while allowing 4,389 more tests to execute. The reduction is consistent with sorting identifying reports that can bypass full minimization because an existing group already contains a minimized representative.

RQ2: Sorting and clustering allow LAYOUTQUICKCHECK to avoid fully minimizing reports that match an existing bug group. In the evaluated Firefox test run, this reduces cumulative post-processing time without a throughput penalty, allowing the fuzzer to execute more test cases.

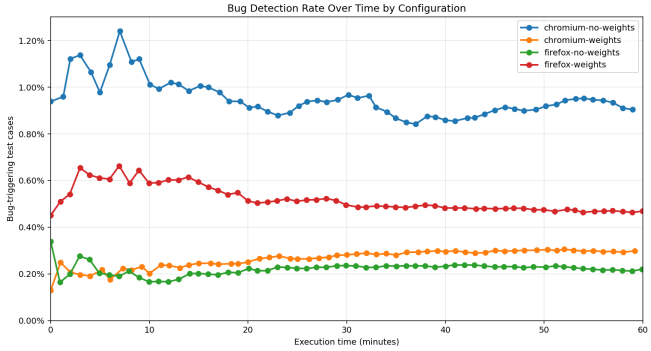
C. RQ3: Diversity of Bug Reports

To assess whether LAYOUTQUICKCHECK produces both recurring and potentially distinct under-invalidation reports, we examine the clustering results of the Firefox sorting and no-weights test runs. Sorting compares each new report with existing groups. A matching report is added to a group, while a report without a match remains a single bug. We compare the number of groups and single bugs over execution time to determine whether clustering captures recurring report patterns without eliminating reports that may require separate triage.

Figure 4 shows the number of bug groups and single bugs as reports are detected for both Firefox configurations.

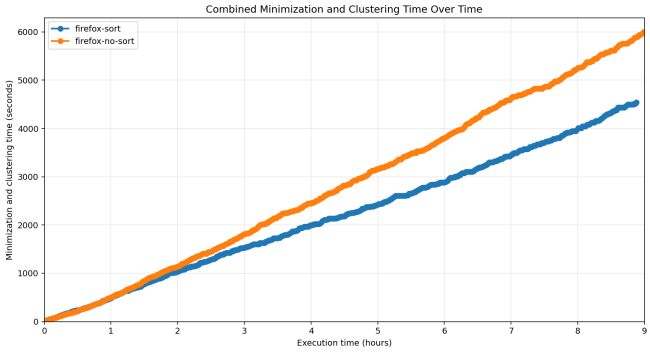


(a) Cumulative bug discovery

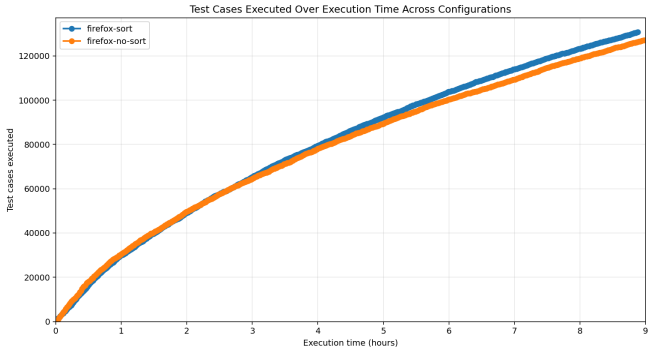


(b) Bug-detection rate

Fig. 2: Bug discovery and bug-detection rates for the weighted and no-weight configurations. Panel (a) shows cumulative bug reports over time; panel (b) shows the fraction of executed tests that trigger the bug oracle.



(a) Combined minimization and clustering time



(b) Test-case throughput

Fig. 3: Firefox sorting and no-sorting configurations over execution time. Panel (a) shows cumulative minimization and clustering time; panel (b) shows cumulative test cases executed.

Outcomes: In the Firefox sorting test run, LAYOUTQUICKCHECK recorded 137 reports after one hour, organized into 20 bug groups and 21 single bugs. At the final 32,040-second snapshot (8 hours and 54 minutes), it recorded 565 reports, 46 bug groups, and 37 single bugs. In the Firefox no-weights test run, LAYOUTQUICKCHECK recorded 63 reports, 9 groups, and 34 single bugs after one hour. Its final 33,060-second snapshot (9 hours and 11 minutes) contains 237 reports, 29 bug groups, and 91 single bugs. Figure 4 shows that both configurations produce grouped and single reports over time. The results indicate that the fuzzer repeatedly reaches related report patterns while continuing to produce reports that do not match an existing group.

Group and single-bug counts are clustering snapshots, not independently confirmed counts of root-cause bugs. In particular, a later report may merge into an existing group, so these counts can change as the test run progresses and require manual validation before being treated as unique bugs.

RQ3: LAYOUTQUICKCHECK's clustering separates repeated Firefox reports into bug groups while retaining unmatched reports as single bugs for further triage. Across the post-fix Firefox test runs, the output contains both recurring report patterns and reports that may represent distinct under-invalidation behaviors.

VI. DISCUSSION & THREATS TO VALIDITY

Improving Runtime Throughput:

Supporting Other Browsers:

(Other limitation 1)

(Other limitation 2)

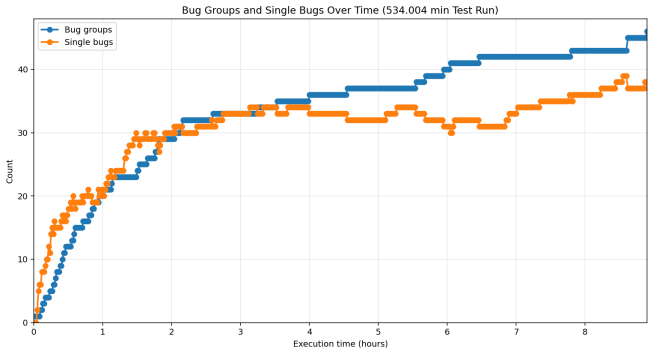
VII. CONCLUSION

ACKNOWLEDGMENTS

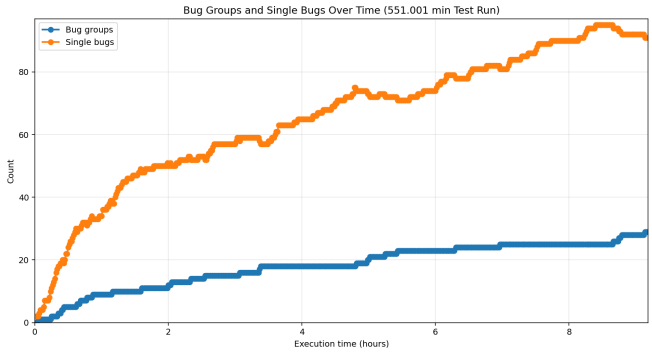
REFERENCES

[1] F. Montenegro, "The state of workforce security: Key insights for it and security leaders," Palo Alto Networks, Tech. Rep., Jan. 2025.

[2] S. Song, J. Hur, S. Kim, P. Rogers, and B. Lee, "R2Z2: Detecting rendering regressions in web browsers through differential fuzz testing," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. ACM, 2022.



(a) Firefox weights



(b) Firefox no weights

Fig. 4: Bug-group and singleton-bug counts by total detected reports for the Firefox sorting and no-weights test runs.

[3] S. Song and B. Lee, "Metamong: Detecting render-update bugs in web browsers through fuzzing," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '23. ACM, 2023.

[4] S. Kim, Y. M. Kim, J. Hur, S. Song, G. Lee, and B. Lee, "FuzzOrigin: Detecting UXSS vulnerabilities in browsers through origin fuzzing," in *Proceedings of the 31st USENIX Security Symposium*, ser. USENIX Security '22. USENIX Association, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/kim

[5] Chromium, "Issue 497183437," Chromium Issue Tracker. [Online]. Available: https://issues.chromium.org/issues/497183437

[6] ——, "Issue 379886422 resources," Chromium Issue Tracker. [Online]. Available: https://issues.chromium.org/issues/379886422/resources

[7] Mozilla, "Bug 1452426," Mozilla Bugzilla. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=1452426

[8] ——, "Bug 539356," Mozilla Bugzilla. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=539356

[9] WebKit, "Bug 233421," WebKit Bugzilla. [Online]. Available: https://bugs.webkit.org/show_bug.cgi?id=233421

[10] ——, "Bug 97801," WebKit Bugzilla. [Online]. Available: https://www2.webkit.org/show_bug.cgi?id=97801

[11] I. Kilpatrick and K. Ishii, "RenderingNG deep-dive: LayoutNG," Chrome for Developers, 2021, last updated October 8, 2021. [Online]. Available: https://developer.chrome.com/docs/chromium/layoutng?hl=en

[12] MozillaWiki, "Gecko: Reflow refactoring," MozillaWiki, 2006, landed December 7, 2006. [Online]. Available: https://wiki.mozilla.org/Gecko:Reflow_Refactoring

[13] The Chromium Projects, "Blink: Rendering engine," The Chromium Projects, accessed June 2, 2026. [Online]. Available: https://www.chromium.org/blink/

[14] Mozilla, "Gecko," Firefox Source Docs, accessed June 2, 2026. [Online]. Available: https://firefox-source-docs.mozilla.org/overview/gecko.html

[15] WebKit, "Webkit," WebKit, accessed June 2, 2026. [Online]. Available: https://webkit.org/

[16] Open Web Advocacy, "Break google's search monopoly without breaking the web," Open Web Advocacy, Apr. 2025, published April 23, 2025. [Online]. Available: https://open-web-advocacy.org/blog/break-googles-search-monopoly-without-breaking-the-web/

[17] M. Kosaka, "Inside look at modern web browser, part 3," Chrome for Developers, 2018. [Online]. Available: https://developer.chrome.com/blog/inside-browser-part3

[18] MDN Web Docs, "Web performance," MDN Web Docs, accessed June 2, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Performance

[19] Mozilla, "Fuzzing," Firefox Source Docs, accessed June 2, 2026. [Online]. Available: https://firefox-source-docs.mozilla.org/tools/fuzzing/index.html

[20] P. Ventuzelo, "Practical web browser fuzzing," Ringzer0 Training, 2023. [Online]. Available: https://ringzer0.training/practical-web-browser-fuzzing/

[21] I. Araoz Severiche, "Web fuzzing: A practical testing methodology," Medium, Feb. 2026, published February 7, 2026. [Online]. Available: https://iaraoz.medium.com/web-fuzzing-a-practical-testing-methodology-1f24198e2ddc

[22] Mozilla, "Performance best practices for firefox front-end engineers," Firefox Source Docs, accessed June 2, 2026. [Online]. Available: https://firefox-source-docs.mozilla.org/performance/bestpractices.html

[23] H. Han, D. Oh, and S. K. Cha, "CodeAlchemist: Semantics-aware code generation to find vulnerabilities in JavaScript engines," in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS '19, 2019.

[24] S. Park, W. Xu, I. Yun, D. Jang, and T. Kim, "Fuzzing JavaScript engines with aspect-preserving mutation," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2020.

[25] H. Peng, Z. Yao, A. Amiri Sani, D. J. Tian, and M. Payer, "GLeeFuzz: Fuzzing WebGL through error message guided mutation," in *Proceedings of the USENIX Security Symposium*, 2023.

[26] O. Draissi, T. Cloosters, D. Klein, M. Rodler, M. Musch, M. Johns, and L. Davi, "Wemby's web: Hunting for memory corruption in WebAssembly," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, 2025.

[27] Y. Zeng, Y. Wu, X. Lu, and C. Zhang, "RTFuzz: Fuzzing browsers via efficient render tree mutation," *Computers & Security*, vol. 161, p. 104756, 2026.

[28] C. Zhou, Q. Zhang, B. Qian, and Y. Jiang, "JANUS: Detecting rendering bugs in web browsers via visual delta consistency," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, ser. ICSE '25. IEEE, 2025.

[29] P. Panchekha and C. Harrelson, *Reusing Previous Computations*. Oxford University Press, 2024, pp. 443–494. [Online]. Available: https://browser.engineering/invalidation.html

[30] J. Liu, Y. Chen, E. Atkinson, Y. Feng, and R. Bodik, "Conflict-driven synthesis for layout engines," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 132:1–132:22, 2023.

[31] A. Zeller, R. Gopinath, M. Böhme, G. Fraser, and C. Holler, "Reducing failure-inducing inputs," The Fuzzing Book, accessed June 2, 2026. [Online]. Available: https://www.fuzzingbook.org/html/Reducer.html

[32] G. Misherghi and Z. Su, "HDD: Hierarchical delta debugging," in *Proceedings of the 28th International Conference on Software Engineering*, ser. ICSE '06. ACM, 2006.

[33] WebKit, "Test case reduction," WebKit, accessed June 2, 2026. [Online]. Available: https://webkit.org/test-case-reduction/